

AURASeg: Attention-Guided Upsampling with Residual-Assisted Boundary Refinement for Drivable-Area Segmentation

Anonymous WACV Algorithms Track submission

Paper ID *****

Abstract

Accurate drivable-area segmentation requires both stable region prediction and precise localization of free-space boundaries. Existing encoder-decoder models often recover coarse region masks while losing thin structures and floor-obstacle transitions, particularly across indoor, outdoor, and road-scene environments. We propose Attention-Guided Upsampling with Residual-Assisted Boundary Refinement (AURASeg), an encoder-decoder architecture that progressively recovers drivable-area structure and then refines its boundaries. Attention Progressive Upsampling Decoder (APUD) blocks align low-resolution semantic features with high-resolution detail features and combine channel-attended semantic responses with spatially attended detail responses through selective fusion. A Residual Boundary Refinement Module (RBRM) derives Sobel-informed boundary features and injects them into the main stream through a learned residual gate, focusing the refinement on contour errors while preserving interior predictions. A compact ASP-PLite bottleneck supplies multi-scale context. Experiments on CARL-D, the Ground Mobile Robot Perception Dataset (GMRPD), and a custom indoor Gazebo dataset show that AURASeg outperforms strong baselines, with notable gains in boundary metrics. Stepwise ablations further show that APUD provides the largest boundary gain and RBRM supplies a complementary final refinement. Jetson Nano profiling demonstrates fully onboard FP32 inference and characterizes the resulting boundary-quality-throughput trade-off.

1. Introduction

Autonomous robotic navigation relies on semantic segmentation to distinguish traversable ground from obstacles and thereby support safe and efficient motion planning. Despite substantial progress in deep learning-based segmentation, feature downsampling and coarse decoder fusion still blur thin structures and floor-obstacle transitions. These

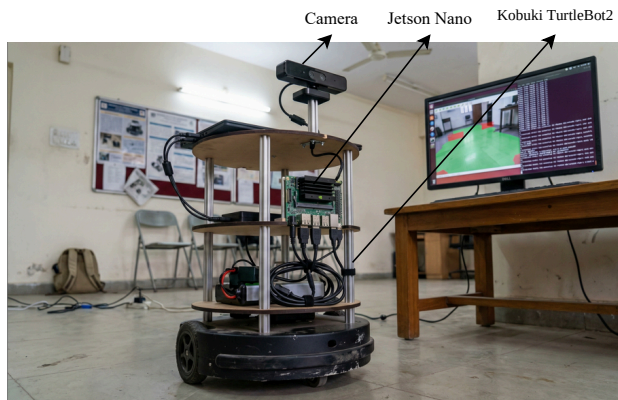


Figure 1. Onboard inference of drivable-area segmentation on a Kobuki TurtleBot2 using an NVIDIA Jetson Nano.

errors are especially consequential for free-space perception because a small contour displacement can convert navigable pixels into false obstacles or remove safety-critical obstacle pixels. Robust drivable-area segmentation therefore requires a model to preserve region consistency while recovering precise boundaries across viewpoint, texture, and illumination changes.

Multi-scale context modules such as Atrous Spatial Pyramid Pooling (ASPP) [3, 4] improve scene-level understanding, while efficient architectures such as BiSeNet [23], Twin-LiteNet [2], and PIDNet [22] retain higher-resolution information for real-time segmentation. Boundary-oriented methods further introduce refinement branches, auxiliary edge supervision, or explicit detail streams [14, 15]. These directions are complementary, but continuously mixing context, detail, and edge features can allow strong semantic responses to overwhelm weak boundary cues or can propagate edge corrections into otherwise stable region interiors.

AURASeg addresses this issue in two clear steps. First, the Attention Progressive Upsampling Decoder (APUD) progressively combines deep semantic features with shallower detail features to recover the main drivable-area structure.

The Residual Boundary Refinement Module (RBRM) then uses the final decoder representation to refine contour regions through a learned gate. This keeps boundary refinement focused on locations that need correction while preserving the main region prediction. ASPPLite supplies compact multi-scale context at the bottleneck.

The method is evaluated across indoor simulation, outdoor ground-robot imagery, and road-scene data, together with controlled component ablations and common-setting Jetson Nano profiling. The principal contributions are:

1. an *Attention Progressive Upsampling Decoder* (APUD) that aligns semantic and detail features, then combines channel-attended semantic responses with spatially attended skip features through selective multiplicative fusion;
2. a *Residual Boundary Refinement Module* (RBRM) that builds Sobel-informed boundary features with a compact encoder-decoder and injects them through a learned residual gate; and
3. an integrated drivable-area segmentation framework that combines multi-scale context, progressive attention-guided decoding, and residual boundary refinement, yielding consistent improvements in region and boundary metrics across indoor, outdoor, and road-scene datasets.

2. Related Work

Efficient encoder-decoder segmentation. Semantic segmentation for autonomous navigation requires high-level context without discarding the spatial detail needed to delineate traversable regions. Ghost-UNet [10] uses an asymmetric encoder-decoder to align low- and high-level features, while residual connections [7] remain a standard mechanism for preserving spatial cues and stabilizing deep networks. Real-time designs pursue stronger accuracy-efficiency trade-offs through bilateral or multi-branch processing. BiSeNet [23] separates spatial and contextual pathways, TwinLiteNet [2] targets compact joint drivable-area and lane segmentation, and PIDNet [22] maintains dedicated detail, context, and boundary branches.

Multi-scale context and attention. ASPP captures multi-scale context through parallel dilated convolutions [3, 4]. FBRNet [15] combines reinforced spatial pooling with cross-scale feature fusion and border refinement, while S2-FPN [6] introduces scale-aware strip-attention connections. Depth-guided free-space segmentation [16] shows how geometric cues can complement appearance information in robotic scenes. Attention mechanisms offer another route to selective feature recovery: Squeeze-and-Excitation (SE) [8] reweights channels, CBAM [19] combines channel and spa-

tial attention, and feature-refinement modules [18] improve semantic retention during decoding.

Boundary-aware and robot-centric segmentation.

Boundary-aware segmentation networks use predict-refine architectures and boundary-sensitive objectives to sharpen fine structures [14]. Street-floor segmentation [9] addresses traversability-related surface understanding for wheeled robots, while D-Flow [12] introduces spatial-temporal modeling for real-time target-area segmentation. RGB-D self-supervision and uncertainty estimation have also been used to generate drivable-area and road-anomaly labels for mobile robots [13, 17].

Positioning of AURASeg. The closest designs share individual ingredients with AURASeg but organize them differently. PIDNet maintains parallel detail, context, and boundary streams; FBRNet couples multi-scale fusion with border reinforcement; and TwinLiteNet prioritizes an extremely compact multi-task architecture. AURASeg instead decomposes single-task drivable-area segmentation sequentially: APUD performs stage-wise semantic/detail reconstruction, after which RBRM predicts a boundary residual and injects only the gated correction into the reconstructed representation. This separation between region recovery and boundary correction is the central architectural distinction of the proposed method.

3. Proposed Method

3.1. Architecture Overview

As depicted in Fig. 2, AURASeg follows a two-stage reconstruction-and-correction architecture for free-space segmentation. A ResNet-18 encoder produces hierarchical features, and ASPPLite aggregates compact multi-scale context at the bottleneck. APUD blocks then reconstruct the region representation progressively by aligning semantic and detail features at successive resolutions. RBRM operates on the final APUD representation as a dedicated correction stage: it estimates boundary-sensitive residual features and injects them through a learned gate. The design therefore preserves the stable region estimate produced by APUD while concentrating additional capacity on contour alignment.

3.2. Encoder Backbone

We use an ImageNet-pretrained ResNet-18 encoder to extract hierarchical features for drivable-area segmentation. ResNet-18 provides a compact and widely established convolutional backbone, while allowing the ablation study to isolate the effects of ASPPLite, APUD, and RBRM without introducing changes associated with a separate backbone redesign. Early stages capture low-level cues (e.g., edges and textures), whereas deeper stages encode high-level semantics. Multi-scale features are forwarded to the decoder through skip connections to recover spatial detail during

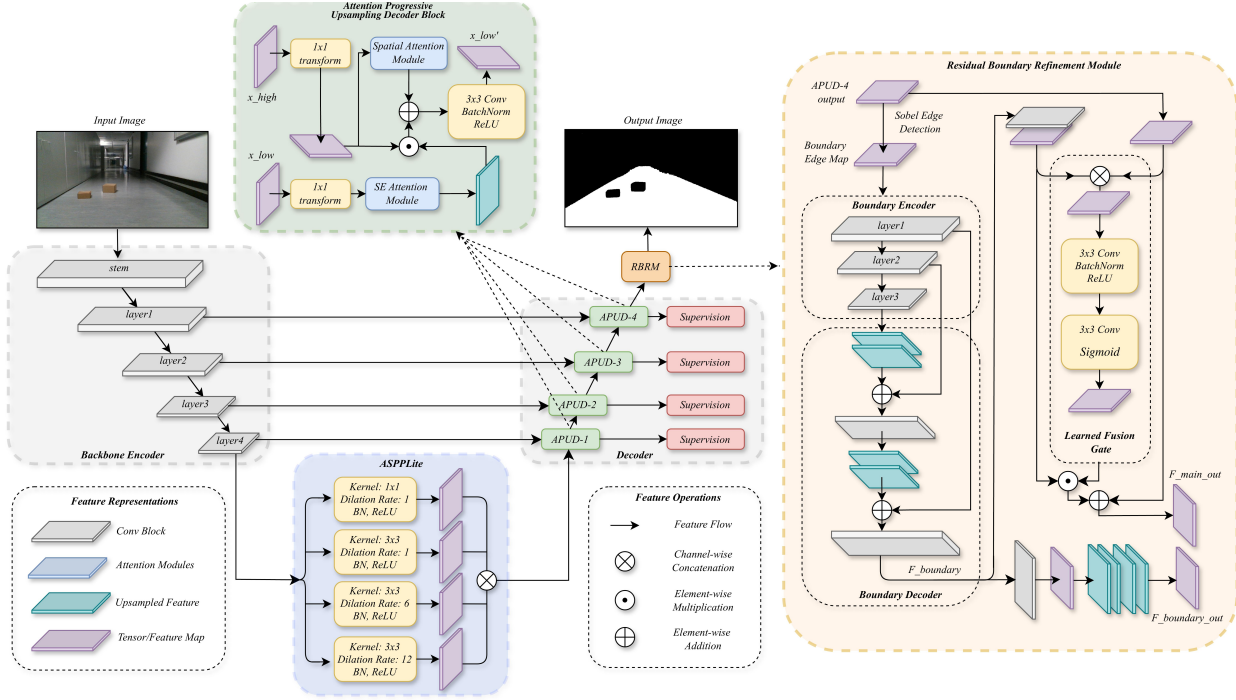


Figure 2. Overview of the proposed encoder-decoder architecture for free-space (drivable-area) segmentation.

progressive upsampling.

3.3. ASPPLite Module

ASPPLite supplies multi-scale contextual cues at the bottleneck with a compact four-branch design. It comprises four parallel branches: one 1×1 projection branch and three 3×3 atrous-convolution branches with dilation rates 1, 6, and 12, each followed by batch normalization and ReLU. The $d=1$ branch preserves local structure, whereas the $d=6$ and $d=12$ branches expand the receptive field to capture mid- and long-range context, improving robustness to texture changes, illumination variation, and scene clutter.

Unlike standard ASPP variants that include additional dilation branches and a global-average-pooling path, ASPPLite omits global pooling to avoid spatial collapse and retain boundary-sensitive information. The branch outputs are concatenated into a multi-scale feature tensor that is forwarded to the decoder.

3.4. Attention Progressive Upsampling Decoder (APUD)

Each APUD block fuses a deep, low-resolution semantic feature map $x_{\text{low}} \in \mathbb{R}^{B \times C_\ell \times H_\ell \times W_\ell}$ with a shallow, high-resolution detail feature map $x_{\text{high}} \in \mathbb{R}^{B \times C_h \times H_h \times W_h}$ to reconstruct segmentation detail progressively. Both inputs are first aligned to a shared embedding dimension C using lightweight 1×1 projections. The semantic branch is then enhanced through Squeeze-and-Excitation (SE) channel

attention [8] and bilinearly upsampled to the skip-feature resolution.

APUD performs selective fusion by multiplying the upsampled semantic signal element-wise with the high-resolution skip feature. This operation acts as a content-dependent gate that suppresses irrelevant textures while retaining boundary-relevant responses. In parallel, a spatial-attention mask [19] refines the skip branch. The gated feature and spatially attended skip feature are added and refined with a 3×3 Conv-BN-Act block, producing $y \in \mathbb{R}^{B \times C \times H_h \times W_h}$:

$$L = T_\ell(x_{\text{low}}), \quad H = T_h(x_{\text{high}}), \quad (1)$$

$$L' = \mathcal{U}(\text{SE}(L)), \quad F = L' \odot H, \quad S = \text{SA}(H), \quad (2)$$

$$y = R(F + S), \quad (3)$$

where $T_\ell(\cdot)$ and $T_h(\cdot)$ are 1×1 Conv-BN-Act projections, $\mathcal{U}(\cdot)$ denotes bilinear upsampling to (H_h, W_h) , $\text{SE}(\cdot)$ is channel attention, $\text{SA}(\cdot)$ is spatial attention, $\odot$ denotes element-wise multiplication, and $R(\cdot)$ is a 3×3 Conv-BN-Act refinement.

3.5. Residual Boundary Refinement (RBRM)

RBRM is placed after the final APUD stage to improve boundary precision in the predicted free-space mask. Although APUD progressively reconstructs features, boundary errors can remain near thin structures or strong texture transitions (e.g., floor-wall boundaries and obstacle contours).

RBRM introduces an explicit boundary-focused branch and merges its cues into the main decoder stream through gated residual fusion.

Let the APUD output be $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$, with $C=256$ at $H/4$ resolution in our setting. RBRM first extracts edge-aware features using a lightweight boundary head. The main feature map is projected from 256 to 64 channels, passed through fixed per-channel Sobel filters to obtain horizontal and vertical responses, and fused by a small learnable convolution to form an edge-sensitive tensor $\mathbf{E} \in \mathbb{R}^{64 \times H \times W}$.

A compact encoder-decoder pathway then processes these features. The boundary encoder applies strided convolutions to obtain multi-level representations ($\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3$), and the decoder reconstructs them through bilinear upsampling. At each stage, the upsampled decoder features are concatenated with the corresponding encoder features and refined by convolution blocks, yielding boundary features $\mathbf{B} \in \mathbb{R}^{64 \times H \times W}$. The boundary feature map is projected to match the main-branch dimensionality, $\mathbf{P} = \phi(\mathbf{B}) \in \mathbb{R}^{C \times H \times W}$, and fused through a learned gate:

$$\mathbf{G} = \sigma(\text{Conv}_{1 \times 1}([\mathbf{F}, \mathbf{P}])) , \quad \mathbf{F}_{\text{ref}} = \mathbf{F} + \mathbf{G} \odot \mathbf{P}, \quad (4)$$

where $[\cdot, \cdot]$ denotes channel concatenation, $\sigma(\cdot)$ is the sigmoid function, and $\odot$ is element-wise multiplication. The gate injects boundary cues only where needed while preserving stable interior regions. In addition to the refined features $\mathbf{F}_{\text{ref}}$ used by the final segmentation head, RBRM outputs an auxiliary boundary map through a 1×1 prediction layer on $\mathbf{B}$ to encourage boundary-aware learning.

3.6. Multi-Loss Supervision and Training

As summarized in Tab. 1, we train AURASeg at an input resolution of 384×640 using an ImageNet-pretrained ResNet-18 encoder. We optimize the network with AdamW and differential learning rates: 1×10^{-4} for the encoder and 1×10^{-3} for the decoder-side modules (ASPPLite, APUD, RBRM, and the segmentation head). A cosine-annealing schedule is applied for up to 50 epochs with early stopping (patience 10) based on validation performance. Training uses a batch size of 8 and mixed precision (FP16/AMP) on a single NVIDIA GeForce RTX 5060 GPU. All baseline models are trained with the same input resolution, augmentation pipeline, optimizer family, batch size, and evaluation protocol. Architecture-specific components are retained for each baseline, while the data and evaluation settings remain controlled across methods.

AURASeg is trained using three complementary supervision signals: (i) region loss on the main prediction, (ii) deep supervision on intermediate APUD outputs, and (iii) boundary-specific loss on the RBRM output. Let Y denote the ground-truth mask and $\hat{Y}$ the main logits, resized to the ground-truth resolution before loss computation. Let $\hat{Y}^{(k)}$ denote the k -th auxiliary APUD logits for $k \in \{1, \dots, 4\}$,

Table 1. Training configuration for AURASeg.

Hyperparameter	Value
<i>Optimization and schedule</i>	
Input resolution	384×640
Encoder backbone	ResNet-18
Optimizer	AdamW ($\beta_1=0.9, \beta_2=0.999$)
Weight decay	0.01
Learning rate (encoder/decoder)	$1 \times 10^{-4} / 1 \times 10^{-3}$
Scheduler	Cosine annealing
Epochs	50 (early-stopping patience: 10)
Batch size	8
Mixed precision	FP16 (AMP)
<i>Loss functions</i>	
Main loss (region)	$0.5 \mathcal{L}_{\text{Focal}} + 0.5 \mathcal{L}_{\text{Dice}}$
Boundary loss	$0.2 \mathcal{L}_{\text{BCE}}$
Auxiliary loss	$0.1 (\mathcal{L}_{\text{Focal}} + \mathcal{L}_{\text{Dice}}) \times 4$
Augmentation	Prob. Parameters
Horizontal flip	0.5 –
Geometric (shift/scale/rotate)	0.5 shift = 0.1, scale = 0.1, rotation = $\pm 15^\circ$
Brightness and contrast	0.3 $\Delta b = \pm 0.2, \Delta c = \pm 0.2$
Gaussian noise	0.2 variance range (10, 50)
Normalization	1.0 ImageNet mean and standard deviation

also resized to the ground-truth resolution. The total objective is

$$\mathcal{L} = \mathcal{L}_{\text{main}} + \lambda_{\text{bnd}} \mathcal{L}_{\text{BCE}} + \sum_{k=1}^4 \lambda_{\text{aux}} \mathcal{L}_{\text{aux}}^{(k)}, \quad (5)$$

where $\lambda_{\text{bnd}} = 0.2$ and $\lambda_{\text{aux}} = 0.1$ (Tab. 1).

Main region loss. We combine Focal loss and Dice loss to balance robustness to class imbalance with overlap-driven optimization:

$$\mathcal{L}_{\text{main}} = 0.5 \mathcal{L}_{\text{Focal}}(\hat{Y}, Y) + 0.5 \mathcal{L}_{\text{Dice}}(\hat{Y}, Y). \quad (6)$$

Auxiliary deep supervision. Each intermediate APUD output is supervised using the same region criterion:

$$\mathcal{L}_{\text{aux}}^{(k)} = \mathcal{L}_{\text{Focal}}(\hat{Y}^{(k)}, Y) + \mathcal{L}_{\text{Dice}}(\hat{Y}^{(k)}, Y). \quad (7)$$

Boundary loss for RBRM. RBRM predicts a boundary-logit map $\hat{B}$, which is upsampled to the ground-truth resolution. The target B is derived from Y using a thin edge band (e.g., a morphological gradient). Let $p_i = \sigma(\hat{B}_i)$. The boundary loss is

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \left(B_i \log p_i + (1 - B_i) \log(1 - p_i) \right), \quad (8)$$

where N is the number of pixels. This objective promotes region consistency and sharp boundary alignment in robotic drivable-area segmentation.

Table 2. Datasets used for drivable-area segmentation.

Dataset	Environment	Train	Val.	Test	Total
Gazebo (ours)	Indoor laboratory corridors (simulation)	2483	294	420	3197
GMRPD [17]	Outdoor sidewalks, plazas, and squares	3000	360	536	3896
CARL-D [1]	On-road scenes from a driver-view camera	9000	2400	2300	14700

Table 3. Ablation study using a ResNet-18 backbone.

Variant	ASPPLite	APUD	RBRM	Params. (M)	Acc.	BF1
V1: Base (ResNet-18)				14.61	0.9928	0.7804
V2: V1 + ASPPLite	✓			17.98	0.9928	0.7931
V3: V2 + APUD	✓	✓		18.21	0.9943	0.8224
V4: V3 + RBRM (proposed)	✓	✓	✓	23.30	0.9946	0.8504

4. Results

4.1. Datasets

As summarized in Tab. 2, we evaluate drivable-area segmentation on three datasets: (i) a synthetic indoor Gazebo dataset collected for robot navigation, (ii) the full GMRPD dataset [17], which contains 3,896 real RGB-D images from outdoor traversal scenes for ground robots (*e.g.*, sidewalks and plazas), and (iii) CARL-D [1], which contains road-scene imagery for autonomous-driving segmentation. Only the RGB images and drivable-area labels are used in our experiments.

The Gazebo and GMRPD partitions are jointly treated as the **MIX** split to evaluate free-space segmentation under combined simulated and real robotic conditions. The reported MIX numbers use the combined validation partitions, while CARL-D is handled separately and reported on its test partition. GMRPD is partitioned into 3,000 training, 360 validation, and 536 test images, preserving the split proportions used in our data pipeline. Together, the datasets cover indoor, outdoor, and road-scene environments and support evaluation of boundary-sensitive free-space perception under distinct visual conditions.

4.2. Performance Analysis

Comparison models. The evaluated methods were selected to cover representative segmentation design families under a common training and evaluation protocol. FCN-R50 [11], PSPNet-R50 [24], DeepLabV3+-R50 [4], and UPerNet-R50 [20] provide established ResNet-based encoder-decoder and multi-scale context baselines. SegFormer-B2 [21] and Mask2Former [5] represent more recent transformer-based segmentation approaches, while PIDNet-L [22] provides a multi-branch architecture designed for efficient high-resolution prediction. Together, these meth-

ods enable comparison against convolutional, transformer-based, boundary-aware, and deployment-oriented segmentation designs.

4.2.1 Ablation Study

Tab. 3 presents a stepwise ablation of the proposed model. Accuracy (Acc.) denotes pixel-wise classification accuracy, and BF1 denotes Boundary F1 computed on boundary pixels extracted using morphological dilation and erosion.

Adding ASPPLite (V2) modestly increases the parameter count while improving BF1 from 0.7804 to 0.7931, indicating stronger multi-scale context aggregation without degrading region-level accuracy. Introducing APUD (V3) yields the largest boundary gain, from 0.7931 to 0.8224, and improves overall accuracy from 0.9928 to 0.9943, highlighting the benefit of attention-guided skip fusion during reconstruction. Finally, adding RBRM (V4) increases BF1 to 0.8504, confirming that explicit boundary-focused refinement complements ASPPLite and APUD.

Fig. 3 provides qualitative comparisons across the same stages. ASPPLite improves global free-space consistency through richer bottleneck context, APUD provides the strongest recovery of fine spatial detail, and RBRM sharpens thin contours and difficult transitions. These visual trends are consistent with the quantitative gains.

4.2.2 Metric Evaluation

Tab. 4 summarizes segmentation performance on the MIX validation set and CARL-D test set at an input resolution of 384×640 . Deployment measurements were collected in 10 W MAXN mode using JetPack 4.6.1 (CUDA 10.2 and cuDNN 8.2), with a batch size of 1 and FP32 inference in PyTorch 1.10. Latency was averaged over 500 forward passes after 50 warm-up iterations.

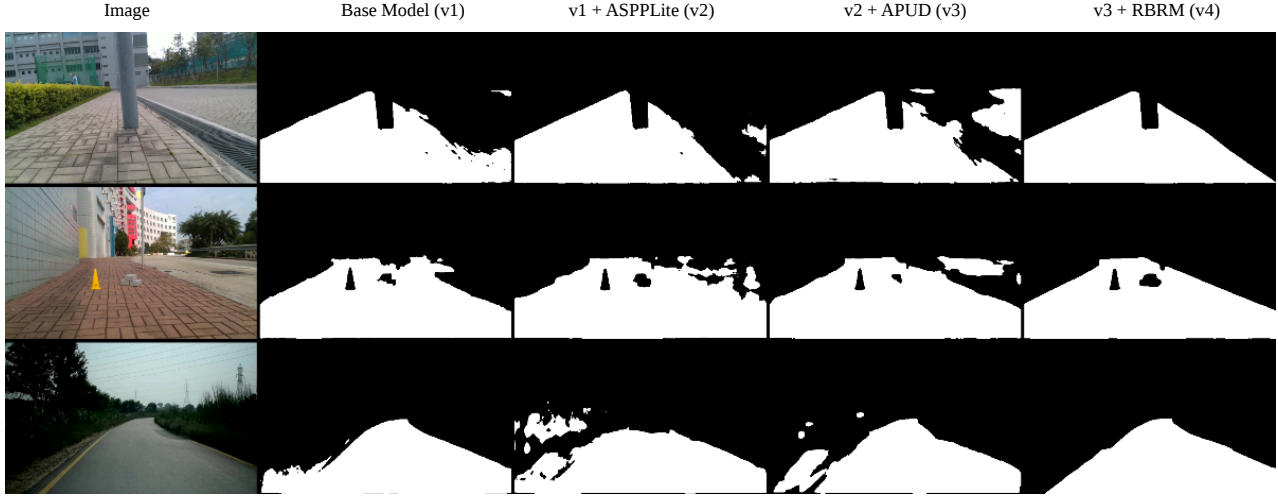


Figure 3. Qualitative comparison across the ablation stages (V1-V4).

Table 4. Performance comparison on the MIX validation set and CARL-D test set.

Model	MIX (Gazebo+GMRPD)						CARL-D					
	IoU	F1	BIoU	BF1	Prec.	Rec.	IoU	F1	BIoU	BF1	Prec.	Rec.
FCN-R50 [11]	0.9857	0.9928	0.6502	0.7789	0.9919	0.9936	0.7735	0.8723	0.0298	0.0448	0.8457	0.9006
PSPNet-R50 [24]	0.9870	0.9935	0.7639	0.8589	0.9941	0.9929	0.7835	0.8786	0.0422	0.0624	0.8594	0.8987
DeepLabV3+-R50 [4]	0.9875	0.9937	0.7799	0.8700	0.9938	0.9936	0.8012	0.8896	0.0416	0.0597	0.8481	0.9354
UPerNet-R50 [20]	0.9879	0.9939	0.7863	0.8738	0.9948	0.9931	0.7965	0.8868	0.0240	0.0371	0.8333	0.9475
SegFormer-B2 [21]	0.9885	0.9942	0.7763	0.8683	0.9958	0.9927	0.7675	0.8685	0.0184	0.0316	0.8613	0.8758
Mask2Former [5]	0.9881	0.9940	0.7740	0.8661	0.9955	0.9925	0.7721	0.8714	0.0183	0.0298	0.8284	0.9190
PIDNet-L [22]	0.9835	0.9917	0.6334	0.7656	0.9912	0.9922	0.7979	0.8876	0.0408	0.0616	0.8580	0.9192
AURASeg (Ours)	0.9897	0.9948	0.8124	0.8905	0.9959	0.9937	0.8041	0.8914	0.0484	0.0683	0.8534	0.9330

Best results for each metric within a dataset are shown in bold. Boundary metrics use $k=2$ dilation iterations. Because CARL-D uses coarse polygon annotations, its boundary scores are not directly comparable with those on MIX (see Sec. 4.2.2).

Table 5. On-device deployment comparison on an NVIDIA Jetson Nano (4 GB).

Model	Params. (M)	GFLOPs (approx.)	Peak memory (MB)	Latency (ms)	FPS
FCN-R50 [11]	35.31	139.25	265.5	843.2	1.19
DeepLabV3+-R50 [4]	26.68	34.60	230.9	248.7	4.02
UPerNet-R50 [20]	37.28	71.93	478.2	523.4	1.91
SegFormer-B2 [21]	24.72	19.86	285.4	418.6	2.39
PIDNet-L [22]	36.94	32.35	192.3	210.3	4.76
AURASeg	23.30	134.7	415.7	481.0	2.08

FPS is computed as $1000/\text{latency (ms)}$. GFLOPs denotes the approximate operation count for one 384×640 forward pass and is independent of measured latency.

In addition to segmentation quality, Tab. 5 reports parameter count, approximate GFLOPs, peak memory, latency, and throughput on the NVIDIA Jetson Nano. AURASeg uses the fewest parameters among the compared models and records a lower measured latency than FCN-R50 and UPerNet-R50

under the common FP32 deployment protocol, while attaining the strongest boundary metrics in Tab. 4. Despite its higher operation count relative to UPerNet-R50, its latency is lower because runtime also depends on operator composition, memory access, kernel scheduling, and hardware uti-

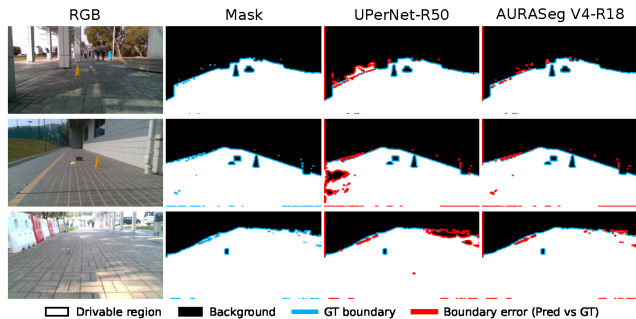


Figure 4. Qualitative boundary comparison between UPerNet-R50 and AURASeg. Red denotes boundary errors relative to the ground-truth boundary (blue).

lization. SegFormer-B2 has the lowest theoretical operation count, whereas PIDNet-L achieves the highest throughput; consequently, approximate GFLOPs alone do not determine runtime on the Jetson Nano. The Jetson study demonstrates fully onboard FP32 execution under a common measurement protocol and characterizes the accuracy–throughput trade-off of high-resolution APUD and RBRM processing.

Boundary Evaluation Protocol. We compute Boundary IoU (BIOU) and Boundary F1 (BF1) by extracting a boundary band using a morphological gradient with a 3×3 kernel, followed by two dilation iterations ($k=2$). This tolerance accommodates minor spatial misalignment while penalizing larger boundary errors. The same protocol is applied to all methods and both datasets.

MIX Dataset (Gazebo+GMRPD). AURASeg achieves the strongest boundary localization, with BIOU=0.8124 and BF1=0.8905. Relative to the strongest baseline, UPerNet-R50 (BIOU=0.7863, BF1=0.8738), these values represent improvements of 3.3% and 1.9%, respectively. Region-level performance also remains strong ($\text{IoU}_{\text{drv}}=0.9897$, F1=0.9948), showing that improved boundaries do not compromise interior-region consistency.

CARL-D Dataset. On CARL-D, AURASeg achieves the best overall IoU, F1, BIOU, and BF1: $\text{IoU}_{\text{drv}}=0.8041$, F1=0.8914, BIOU=0.0484, and BF1=0.0683. The lower absolute boundary scores arise from CARL-D’s smooth polygon annotations, which produce far fewer boundary pixels (~ 200 versus $\sim 5,000$ on MIX). AURASeg improves BIOU by 14.7% and BF1 by 9.5% over the next-best method, PSPNet-R50. These results show that explicit boundary refinement remains beneficial even under annotation-limited evaluation and yields consistent gains across robot-centric and road-scene datasets.

Fig. 4 compares RGB inputs, ground truth, and predictions on representative samples. AURASeg produces cleaner and better-aligned free-space contours than UPerNet-R50, particularly near thin obstacles, floor–wall transitions, cluttered structures, and visually ambiguous free-space regions.

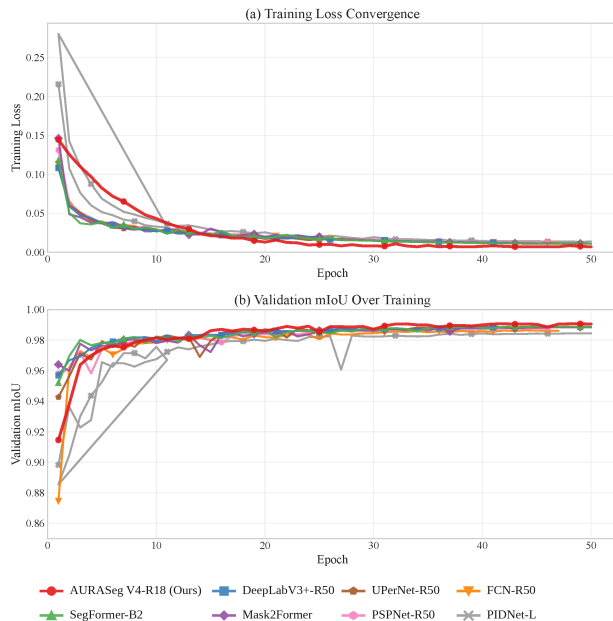


Figure 5. Training-loss and validation-IoU curves for AURASeg and the baseline models.

As shown in Fig. 5, AURASeg also converges rapidly and maintains a stable validation plateau under the unified training protocol.

4.3. Discussion

The ablation results clarify how the components contribute within AURASeg. APUD produces the largest single BF1 gain, indicating that progressive semantic/detail fusion is the main driver of spatial recovery. RBRM then yields a further improvement without reducing region accuracy, showing that the gated residual branch sharpens boundaries while preserving the main region prediction. The full model consequently improves both overlap and boundary metrics rather than optimizing one at the expense of the other.

Performance trends differ in magnitude across the two evaluation settings because CARL-D uses coarse polygon annotations that produce fewer and smoother boundary pixels than MIX. Its absolute boundary scores should therefore be interpreted within the CARL-D benchmark, while the relative ranking across methods remains informative. The Jetson Nano measurements further show that AURASeg maintains a moderate parameter count, while its high-resolution spatial operations account for most of the computational cost. Together, these results characterize the accuracy–computation trade-off associated with boundary-sensitive refinement.

5. Conclusion

We presented AURASeg, an encoder–decoder architecture that combines ASPPLite, APUD, and RBRM for

drivable-area segmentation. APUD progressively fuses semantic and detail features to recover spatial structure, while RBRM applies Sobel-informed gated residual refinement to improve contour quality. Experiments on Gazebo, GM-RPD, and CARL-D demonstrate consistent improvements in boundary metrics while maintaining strong region accuracy, and Jetson Nano profiling verifies fully onboard inference. Overall, AURASeg provides an effective boundary-aware segmentation framework for robot-centric and road-scene perception.

References

- [1] M. A. Butt and F. Riaz. CARL-D: A vision benchmark suite and large-scale dataset for vehicle detection and scene segmentation. *Signal Processing: Image Communication*, 104:116667, 2022. 5
- [2] Q. H. Che, D. P. Nguyen, M. Q. Pham, and D. K. Lam. Twin-LiteNet: An efficient and lightweight model for driveable-area and lane segmentation in self-driving cars. *arXiv preprint arXiv:2307.10705*, 2023. 1, 2
- [3] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L. Yuille. DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *arXiv preprint arXiv:1606.00915*, 2016. 1, 2
- [4] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *European Conference on Computer Vision (ECCV)*, 2018. 1, 2, 5, 6
- [5] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girshick. Masked-attention mask transformer for universal image segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 5, 6
- [6] M. A. M. Elhassan et al. S2-FPN: Scale-aware strip-attention-guided feature pyramid network for real-time semantic segmentation. *arXiv preprint arXiv:2206.07298*, 2022. 2
- [7] Kaifeng He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 2
- [8] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 2, 3
- [9] J. Hyun, S. Woo, S. Lee, Y. Kim, and S. Ko. Street floor segmentation for a wheeled mobile robot. *IEEE Access*, 2022. 2
- [10] I. A. Kazerouni, G. Dooly, and D. J. F. Toal. Ghost-UNet: An asymmetric encoder-decoder architecture for semantic segmentation from scratch. *IEEE Access*, 2021. 2
- [11] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015. 5, 6
- [12] Wentao Lu and Claude Sammut. D-Flow: A real-time spatial-temporal model for target-area segmentation. *arXiv preprint arXiv:2111.04525*, 2021. 2
- [13] Toan-Khoa Nguyen, Phuc Thanh-Thien Nguyen, Dai-Dong Nguyen, and Chung-Hsien Kuo. Effective free-driving region detection for mobile robots by uncertainty estimation using RGB-D data. *Sensors*, 22(13):4751, 2022. 2
- [14] Xuebin Qin, Deng-Ping Fan, Chenyang Huang, Cyril Diagne, Zichen Zhang, Adrià Cabeza Sant'Anna, Albert Suàrez, Martin Jagersand, and Ling Shao. Boundary-aware segmentation network for mobile and web applications. *arXiv preprint arXiv:2101.04704*, 2021. 1, 2
- [15] S. Qu, Z. Wang, J. Wu, and Y. Feng. FBRNet: A feature fusion and border refinement network for real-time semantic segmentation. *Pattern Analysis and Applications*, 27:22, 2024. 1, 2
- [16] Christos Sevastopoulos, Joey Hussain, Stasinios Konstantopoulos, Vangelis Karkaletsis, and Fillia Makedon. Depth-guided free-space segmentation for a mobile robot. *arXiv preprint arXiv:2311.01966*, 2023. 2
- [17] Hengli Wang, Yuxiang Sun, and Ming Liu. Self-supervised drivable area and road anomaly segmentation using RGB-D data for robotic wheelchairs. *IEEE Robotics and Automation Letters*, 4(4):4386–4393, 2019. 2, 5
- [18] Z. Wang, X. Guo, S. Wang, P. Zheng, and L. Qi. A feature refinement module for light-weight semantic segmentation network. *arXiv preprint arXiv:2412.08670*, 2024. 2
- [19] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon. CBAM: Convolutional block attention module. In *European Conference on Computer Vision (ECCV)*, 2018. 2, 3
- [20] T. Xiao, Y. Liu, B. Zhou, Y. Jiang, and J. Sun. Unified perceptual parsing for scene understanding. In *European Conference on Computer Vision (ECCV)*, 2018. 5, 6
- [21] E. Xie et al. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 5, 6
- [22] Jiacong Xu, Zixiang Xiong, and Shankar P. Bhattacharyya. PIDNet: A real-time semantic segmentation network inspired by PID controllers. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19529–19539, 2023. 1, 2, 5, 6
- [23] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang. BiSeNet: Bilateral segmentation network for real-time semantic segmentation. *arXiv preprint arXiv:1808.00897*, 2018. 1, 2
- [24] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia. Pyramid scene parsing network. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 5, 6